Pronóstico 2027 — ATUS ZMM

El modelo jerárquico de `modelo_jerarquico.qmd`, puesto a pronosticar. Ahí se entrenó con 2019-2023 y se validó en 2024; aquí se **reajusta con los seis años** y se proyecta a 2027, tres años después del último dato. El producto es un GeoParquet con el pronóstico por celda — mediana, intervalo y probabilidad de estar en el 5 % superior — listo para mapear o cruzar con otras capas.

Fuente: `data/processed/atus_zmm_limpio_geo.parquet`, particionado con `geostats.espacial.celdas`.

Qué hay aquí

§	Tema
1	Reajuste con 2019-2024 y convergencia
2	Dos formas de proyectar: persistencia y régimen
3	El total metropolitano: cómo crece la incertidumbre con el horizonte
4	Por municipio
5	Por celda: el mapa de 2027 y dónde está seguro el modelo
6	El producto: `pronostico_2027_geo.parquet`

> **Qué puede y qué no puede decir este pronóstico.** El modelo no tiene tendencia: la deriva municipal se probó en el notebook anterior y los datos la encogieron a cero. Su único mecanismo de cambio en el tiempo es la caminata aleatoria del régimen municipal, con innovaciones *t* de colas pesadas — y como $E[e^T] = \infty$, admitir esas innovaciones deja al pronóstico sin media. Por eso el **pronóstico central es por persistencia**: el nivel de 2024 de cada municipio, con su incertidumbre a posteriori, por el perfil estable de cada celda. El escenario con cambio de régimen se presenta aparte, solo con cuantiles (que sí existen), como la cota de lo que el modelo no controla. Y lo de siempre: $m_{j,a}$ es *reporte*, no siniestralidad.

```
import warnings
warnings.filterwarnings("ignore")

import time
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
```

```
from matplotlib.colors import LinearSegmentedColormap, TwoSlopeNorm, LogNorm
from matplotlib.ticker import NullFormatter
from scipy import stats
import pymc as pm
import arviz as az
from libpysal import weights

from geostats import espacial, rutas

SEMILLA = 42
CALENTAMIENTO, MUESTRAS = 3000, 4000   # mismas que modelo_jerarquico.qmd
rng = np.random.default_rng(SEMILLA)

g = gpd.read_parquet(rutas.ATUS_ZMM_LIMPIO_GEO).to_crs(espacial.UTM_ZMM)
print(f"{len(g):,} registros | PyMC {pm.__version__}")
```

```
379,294 registros | PyMC 6.3.2
```

Estilo de las gráficas

Misma identidad que los notebooks anteriores.

```
# --- Identidad GeoStats ------------------------------------------------------
AZUL     = "#005991"
ROJO     = "#8B2C1A"
OXIDO    = "#B15E2E"
GRAFITO  = "#2C2C2C"
GRIS     = "#F2F2F2"
BLANCO   = "#FFFFFF"
AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]

SECUENCIAL = LinearSegmentedColormap.from_list("geostats_seq", [BLANCO, AZUL])
DIVERGENTE = LinearSegmentedColormap.from_list("geostats_div", [AZUL, GRIS, ROJO])

TITULAR = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO   = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS  = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 16,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,
    "grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
    "xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.frameon": False,
    "legend.fontsize": 8.5, "lines.linewidth": 1.8,
```

```
})

def pulir(ax, ejes=("top", "right", "left")):
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax

def mapa(ax, titulo=None, escala_km=10):
    ax.set_aspect("equal"); ax.grid(False)
    ax.set_xticks([]); ax.set_yticks([])
    for lado in ax.spines.values():
        lado.set_visible(False)
    if titulo:
        ax.set_title(titulo)
    x0, x1 = ax.get_xlim(); y0, y1 = ax.get_ylim()
    bx = x0 + 0.04 * (x1 - x0); by = y0 + 0.04 * (y1 - y0)
    ax.plot([bx, bx + escala_km * 1000], [by, by], color=GRAFITO, lw=2, solid_capstyle="b
    ax.text(bx, by + 0.012 * (y1 - y0), f"{escala_km} km", fontsize=7.5,
            color=GRAFITO, fontfamily="monospace")
    return ax
```

---

1. Reajuste con 2019-2024

El panel es el mismo del notebook anterior — celda × año sobre los 2,270 hexágonos de 500 m — pero ahora **sin partición**: los seis años entran al ajuste. No hay conjunto de prueba porque el objetivo ya no es validar el modelo (eso se hizo en 2024) sino usar toda la información disponible para proyectar. El modelo, los priors y el muestreador son idénticos; solo cambia `N_ANIOS` de 5 a 6.

```
H, W = espacial.celdas(g, lado=500)
assert len(H) == 2270 and len(W.islands) == 0

union = gpd.sjoin(g[["geometry", "ANIO", "NOM_MUN"]], H[["hex", "geometry"]],
                  predicate="within", how="inner")
ANIOS = list(range(2019, 2025))
indice = pd.MultiIndex.from_product([H.hex, ANIOS], names=["hex", "anio"])
p = union.groupby(["hex", "ANIO"]).size().reindex(indice, fill_value=0).rename("y").reset

municipio = union.groupby("hex").NOM_MUN.agg(lambda s: s.mode().iloc[0])
p["municipio"] = p.hex.map(municipio)
MUNICIPIOS = sorted(municipio.unique())
hex_a_i = {h: i for i, h in enumerate(H.hex)}
mun_a_j = {m: j for j, m in enumerate(MUNICIPIOS)}
p["i"] = p.hex.map(hex_a_i)
p["j"] = p.municipio.map(mun_a_j)
```

```
p["a"] = p.anio - 2019
N_HEX, N_MUN, N_ANIOS = len(H), len(MUNICIPIOS), len(ANIOS)

Wb = weights.W(W.neighbors)
Wb.transform = "b"
A = Wb.full()[0]
assert (A == A.T).all()

print(f"{len(p):,} celda-año (2019–2024) | {N_HEX:,} celdas en {N_MUN} municipios")
```

```
13,620 celda-año (2019–2024) | 2,270 celdas en 18 municipios
```

```
with pm.Model() as modelo:
    b0 = pm.Normal("b0", np.log(p.y.mean() + 1), 2)

    s_m = pm.HalfNormal("s_m", 1.5)
    s_rw = pm.HalfNormal("s_rw", 0.5)
    z0 = pm.ZeroSumNormal("z0", sigma=1, shape=N_MUN)
    eps = pm.StudentT("eps", nu=3, mu=0, sigma=1, shape=(N_MUN, N_ANIOS - 1))
    m0 = (s_m * z0)[:, None]
    m = pm.Deterministic("m", pm.math.concatenate([m0, m0 + pm.math.cumsum(s_rw * eps, ax

    alpha = pm.Beta("alpha", 2, 1)
    s_u = pm.HalfNormal("s_u", 1.5)
    u = pm.CAR("u", mu=np.zeros(N_HEX), W=A, alpha=alpha, tau=1 / s_u**2, shape=N_HEX)

    theta = pm.Gamma("theta", 2, 0.1)
    log_mu = pm.Deterministic("log_mu", b0 + m[p.j.values, p.a.values] + u[p.i.values])
    pm.NegativeBinomial("y", mu=pm.math.exp(log_mu), alpha=theta, observed=p.y.values)

    inicio = time.time()
    idata = pm.sample(draws=MUESTRAS, tune=CALENTAMIENTO, chains=4, nuts_sampler="nutpie"
                      random_seed=SEMILLA, progressbar=False)
print(f"4 cadenas × ({CALENTAMIENTO:,} calentamiento + {MUESTRAS:,} muestras) "
      f"en {(time.time() - inicio) / 60:.1f} min")
```

```
NUTS[nutpie]: [b0, s_m, s_rw, z0, eps, alpha, s_u, u, theta]

4 cadenas × (3,000 calentamiento + 4,000 muestras) en 9.3 min
```

Convergencia

Los mismos tres criterios del notebook anterior, sobre las mismas cantidades identificadas ($\log \mu$ y los hiperparámetros). β_0 y σ_m siguen mezclando peor por la razón ya dicha — comparten información con el nivel medio de u — y siguen sin usarse solos en ninguna conclusión.

```
HIPER = ["b0", "s_m", "s_rw", "alpha", "s_u", "theta"]
resumen = az.summary(idata, var_names=HIPER)[["mean", "sd", "ess_bulk", "r_hat"]]
divergencias = int(idata.sample_stats.diverging.sum())
```

```
rhat_logmu = float(az.rhat(idata, var_names=["log_mu"]).log_mu.max())
ess_logmu = float(az.ess(idata, var_names=["log_mu"]).log_mu.min())
print(f"Divergencias: {divergencias}")
print(f"log μ — R̂ máximo {rhat_logmu:.3f}, ESS mínimo {ess_logmu:,.0f} (sobre {len(p):,}
resumen.round(3)
```

```
Divergencias: 1
log μ — R̂ máximo 1.002, ESS mínimo 3,250 (sobre 13,620 cantidades)
```

	mean	sd	ess_bulk	r_hat
b0	1.48	0.13	76	1.04
s_m	1.24	0.25	38	1.10
s_rw	0.286	0.032	223	1.02
alpha	0.916	0.0149	142	1.01
s_u	2.551	0.0454	2286	1.00
theta	12.036	0.395	11268	1.00

Una transición divergente y $\hat{R} < 1.01$ en todas las $\log \mu$. Los hiperparámetros quedan donde estaban con cinco años: $\alpha = 0.92$, $\theta = 12$, $\sigma_{rw} = 0.29$. Como en el notebook anterior, β_0 y σ_m mezclan lento ($\hat{R}$ de 1.04 y 1.10, ESS de 76 y 38): comparten información con el nivel medio de u, y ninguno se usa solo. Añadir 2024 no cambió lo que el modelo cree sobre la estructura; solo actualizó el nivel de cada municipio.

```
post = az.extract(idata)
u_post = post.u.values                          # (hex, draws)
m_post = post.m.values                          # (mun, año, draws)
b0_post = post.b0.values
theta_post = post.theta.values
s_rw_post = post.s_rw.values
S = u_post.shape[1]

# Una fila por celda, con el observado de 2024 al lado.
celda = p[p.anio == 2024].reset_index(drop=True)[["hex", "municipio", "i", "j", "y"]].ren
```

2. Dos formas de proyectar

Para la celda i en el año $2024 + h$, el modelo dice

$$\log \mu_{i,2024+h} = \beta_0 + m_{j(i),2024+h} + u_i, \qquad m_{j,2024+h} = m_{j,2024} + \sigma_{rw} \sum_{k=1}^{h} \varepsilon_{j,k}, \quad \varepsilon \sim t_3.$$

Todo lo que está a la derecha salvo las ε ya tiene distribución a posteriori. La pregunta es qué hacer con ellas:

Escenario	ε	Qué representa	Qué existe
Persistencia	$\equiv 0$	Ningún municipio cambia de régimen entre 2025 y 2027	Media, mediana, cuantiles, CRPS
Régimen	$\sim t_3$, tres por municipio	Cada municipio sigue moviéndose como se movió en 2019-2024	Solo cuantiles

La persistencia es el escenario del notebook anterior, extendido de $h = 1$ a $h = 3$; es el que calibró 2024 al 1 %. El escenario de régimen es la respuesta honesta a "¿y si Santa Catarina vuelve a pasar?": no dice *cuándo* ni *dónde*, pero sí cuánto se abre el intervalo si se admite que puede pasar. Sus medias no existen (la §7 del notebook anterior lo mostró en números), así que aquí se reportan únicamente medianas y percentiles.

```
TOPE = 1e5   # la cola t puede dar λ astronómicos en algún draw; el tope no toca ningún c

def simular(m_h):
    """λ y conteos NB simulados para cada celda y cada draw, dado el nivel municipal m_h
    lam = np.minimum(np.exp(b0_post[None, :] + m_h[celda.j.values, :] + u_post[celda.i.va
    y = rng.negative_binomial(theta_post[None, :], theta_post[None, :] / (theta_post[None
    return lam, y

m_2024 = m_post[:, -1, :]
HORIZONTES = [2025, 2026, 2027]

# Persistencia: el mismo nivel los tres años; se simula una vez por horizonte para la fig
pers = {h: simular(m_2024) for h in HORIZONTES}

# Régimen: innovaciones t acumuladas, un camino por draw y municipio.
eps_sim = rng.standard_t(3, size=(N_MUN, len(HORIZONTES), S))
m_reg = {h: m_2024 + s_rw_post[None, :] * eps_sim[:, :k + 1, :].sum(axis=1) for k, h in e
reg = {h: simular(m_reg[h]) for h in HORIZONTES}

lam_27, y_27 = pers[2027]
lam_27r, y_27r = reg[2027]
print(f"Draws con λ en el tope (régimen 2027): {(lam_27r >= TOPE).mean():.4%}")
```

```
Draws con λ en el tope (régimen 2027): 0.0132%
```

3. El total metropolitano

```
observado = p.groupby("anio").y.sum()
def cuantiles_total(y):
    return np.percentile(y.sum(axis=0), [5, 25, 50, 75, 95])
```

```
tot_p = pd.DataFrame({h: cuantiles_total(pers[h][1]) for h in HORIZONTES}, index=["q05",
tot_r = pd.DataFrame({h: cuantiles_total(reg[h][1]) for h in HORIZONTES}, index=["q05", "

print(f"2024 observado: {observado[2024]:,}   |   ajustado por el modelo: {lam_27.mean(ax
      f"({lam_27.mean(axis=1).sum() / observado[2024] - 1:+.1%})")
print("\nPersistencia — total anual, percentiles del ensamble")
print(tot_p.round(0).astype(int).to_string())
print("\nRégimen — total anual, percentiles del ensamble")
print(tot_r.round(0).astype(int).to_string())
```

```
2024 observado: 71,295   |   ajustado por el modelo: 73,645 (+3.3%)

Persistencia — total anual, percentiles del ensamble
        q05    q25    q50    q75    q95
2025  71356  72686  73621  74576  75992
2026  71397  72692  73622  74572  75956
2027  71364  72698  73644  74607  76014

Régimen — total anual, percentiles del ensamble
        q05    q25    q50     q75     q95
2025  57706  69336  77583   88526  123469
2026  54278  69325  82682  101838  174035
2027  53265  70906  88782  115364  226995
```

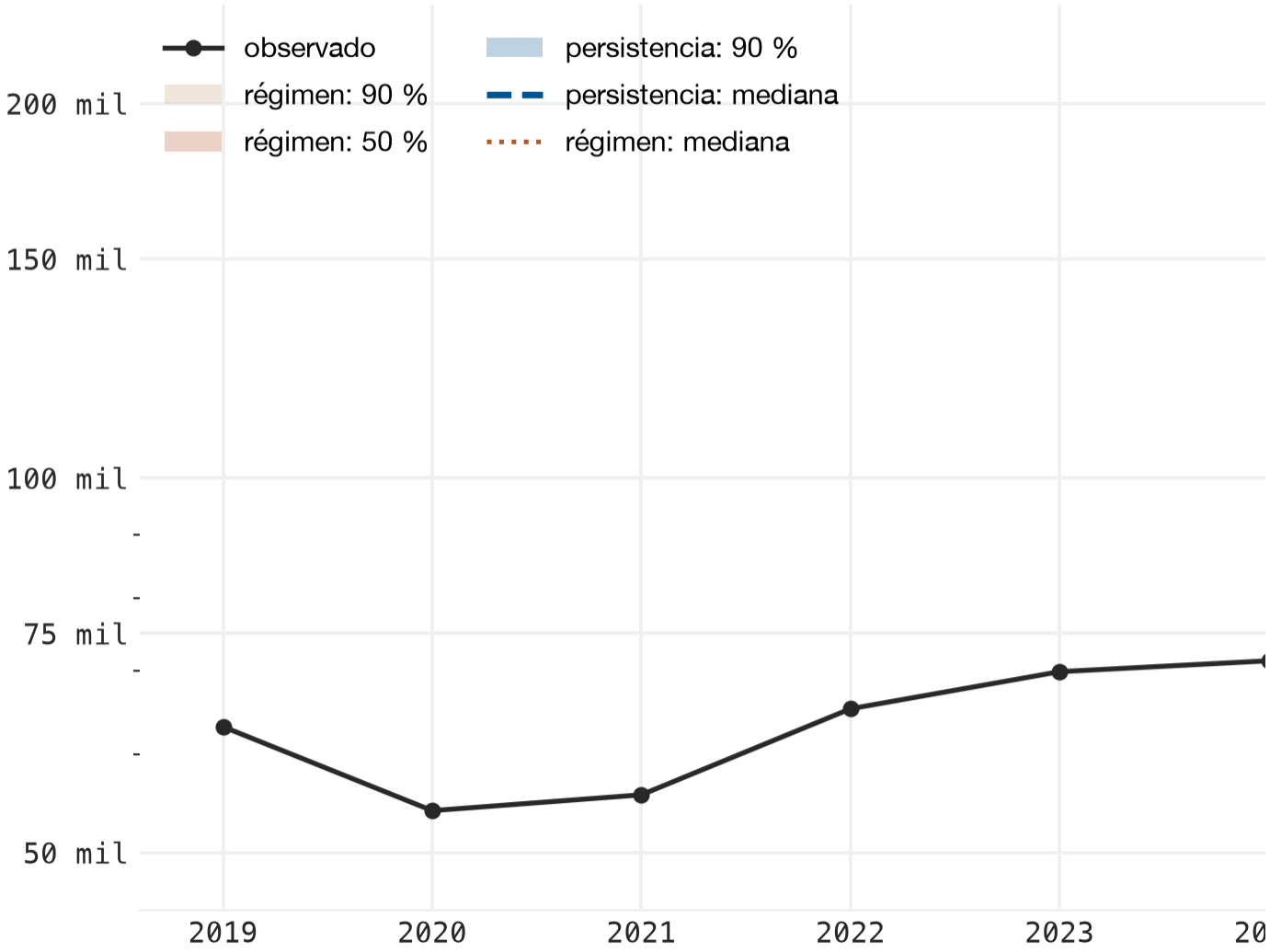
```
fig, ax = plt.subplots(figsize=(8.5, 4.6))
x_obs = observado.index.values
ax.plot(x_obs, observado.values, color=GRAFITO, marker="o", ms=4, lw=1.5, label="observad

xh = np.array([2024] + HORIZONTES)
def banda(df, q_lo, q_hi, color, alpha, etiqueta=None):
    lo = np.r_[observado[2024], df[q_lo].values]; hi = np.r_[observado[2024], df[q_hi].va
    ax.fill_between(xh, lo, hi, color=color, alpha=alpha, linewidth=0, label=etiqueta)

banda(tot_r, "q05", "q95", OXIDO, 0.15, "régimen: 90 %")
banda(tot_r, "q25", "q75", OXIDO, 0.25, "régimen: 50 %")
banda(tot_p, "q05", "q95", AZUL, 0.25, "persistencia: 90 %")
ax.plot(xh, np.r_[observado[2024], tot_p.q50.values], color=AZUL, lw=2, ls="--", label="p
ax.plot(xh, np.r_[observado[2024], tot_r.q50.values], color=OXIDO, lw=1.4, ls=":", label=

ax.set_yscale("log")
ax.set_yticks([50_000, 75_000, 100_000, 150_000, 200_000])
ax.set_yticklabels(["50 mil", "75 mil", "100 mil", "150 mil", "200 mil"])
ax.yaxis.set_minor_formatter(NullFormatter())
ax.set_ylim(45_000, 240_000)
ax.set_xticks(list(range(2019, 2028)))
ax.set_title("Lo que el modelo controla y lo que no")
ax.legend(loc="upper left", ncol=2)
pulir(ax); plt.tight_layout(); plt.show()
```

Lo que el modelo controla y lo que no



Accidentes anuales en la ZMM: observados 2019-2024 y pronóstico 2025-2027. La banda angosta es persistencia (intervalos del 50 y 90 %); la banda ancha, el escenario de régimen. El eje es logarítmico porque el escenario de régimen es multiplicativo.

Tres lecturas:

- **Por persistencia, 2027 es 2024 suavizado.** La mediana del total queda en ~73.6 mil contra 71.3 mil observados en 2024, y el intervalo del 90 % es de apenas ±3 %. Ese 3 % de más es el sesgo de suavizado del CAR sobre las celdas con pocos datos (una celda en cero recibe $\lambda > 0$ de sus vecinas), no una tendencia: el modelo no tiene término de tendencia, y el pronóstico central de 2027 es idéntico al de 2025 y 2026.
- **El intervalo de persistencia es un piso.** Contiene la incertidumbre de los parámetros y la dispersión NB; no contiene la posibilidad de que un municipio cambie de régimen. Es lo que dice la advertencia inicial.
- **El escenario de régimen se abre rápido y hacia arriba.** A tres años el 90 % va de ~53 mil a ~230 mil, y su mediana sube a ~89 mil aunque la de cada celda no se mueva. La asimetría es aritmética: en escala logarítmica la innovación es simétrica, pero un municipio que salta ×5 añade cuatro veces su tamaño al total y uno que cae ÷5 solo resta el 80 %. Con seis años dominados por ampliaciones

de cobertura (2021-2022), el modelo cree que un salto es tan probable como una caída, y en la suma los saltos pesan más. Es un límite del modelo — la caminata aleatoria no tiene reversión ni techo — y por eso la mediana de régimen no es el pronóstico central.

---

4. Por municipio

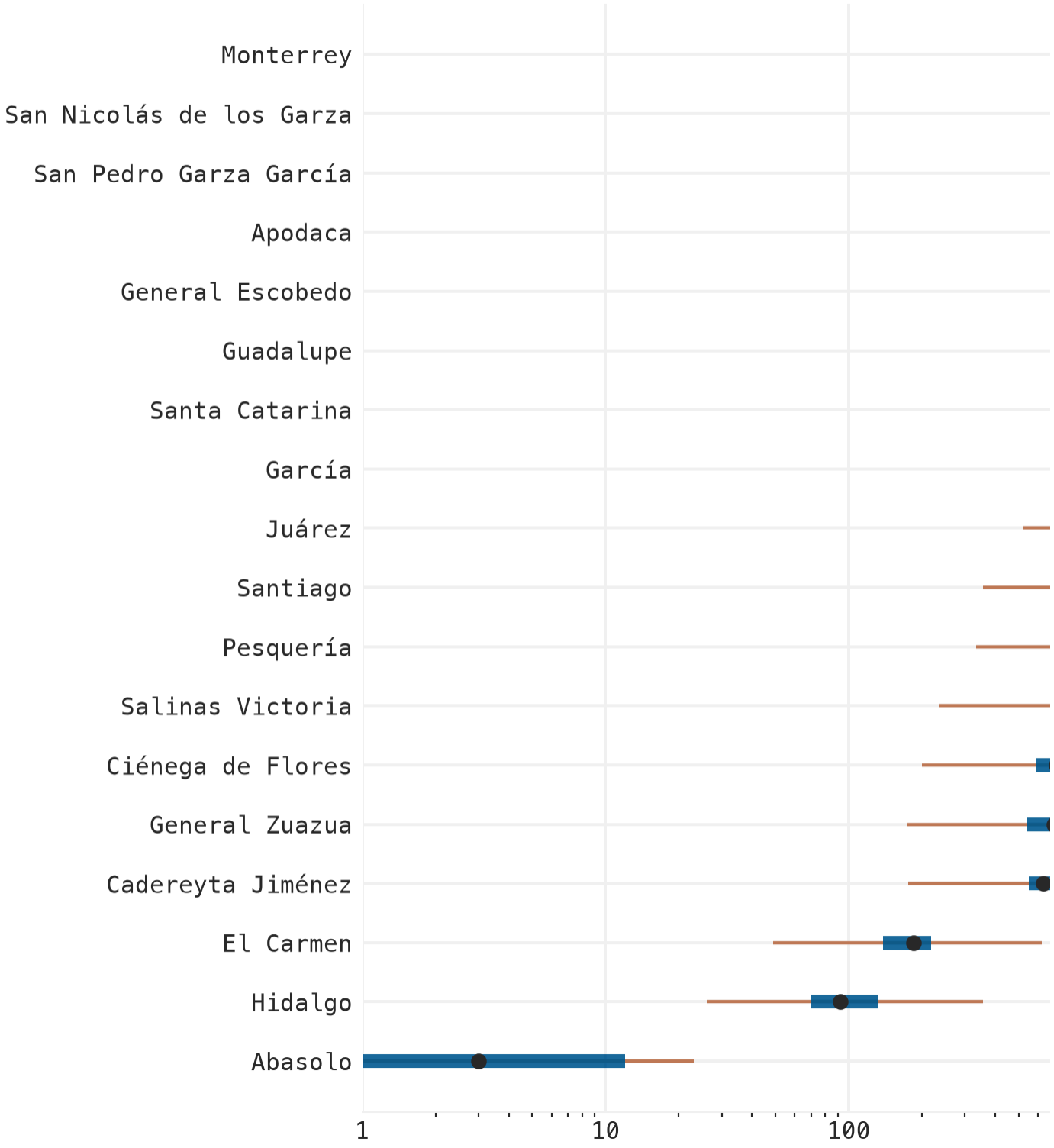
```
def por_municipio(y):
    df = pd.DataFrame(y)
    df["municipio"] = celda.municipio.values
    tot = df.groupby("municipio").sum()
    return pd.DataFrame({"q05": tot.quantile(0.05, axis=1), "q50": tot.median(axis=1), "q

mun_tabla = (celda.groupby("municipio").y_2024.sum().to_frame("2024")
             .join(por_municipio(y_27).add_prefix("pers_"))
             .join(por_municipio(y_27r).add_prefix("reg_")))
mun_tabla["cociente"] = mun_tabla.pers_q50 / mun_tabla["2024"]
mun_tabla = mun_tabla.sort_values("2024", ascending=False)
mun_tabla.style.format({c: "{:,.0f}" for c in mun_tabla.columns if c != "cociente"} | {"c
```

	2024	pers_q05	pers_q50	pers_q95	reg_q05	reg_q50	reg_q95	cociente
municipio								
Monterrey	33,116	31,887	33,702	35,629	9,573	33,636	115,951	1.02
San Nicolás de los Garza	9,226	9,077	9,915	10,830	2,864	9,934	35,241	1.07
San Pedro Garza García	6,878	6,077	6,775	7,543	1,957	6,830	24,100	0.99
Apodaca	3,618	3,466	3,740	4,028	1,052	3,730	13,244	1.03
General Escobedo	3,422	3,296	3,591	3,910	1,044	3,597	12,795	1.05
Guadalupe	2,770	2,640	2,873	3,120	810	2,873	9,981	1.04
Santa Catarina	2,747	2,711	3,087	3,546	886	3,086	11,230	1.12
García	2,394	2,360	2,604	2,872	729	2,625	9,555	1.09
Juárez	1,743	1,641	1,813	1,999	517	1,832	6,243	1.04
Santiago	1,153	1,092	1,230	1,388	356	1,238	4,326	1.07
Pesquería	1,092	1,019	1,156	1,309	334	1,166	4,087	1.06
Salinas Victoria	819	710	807	914	234	807	2,849	0.99
Ciénega de Flores	713	592	701	833	200	700	2,501	0.98
General Zuazua	695	537	648	777	173	638	2,294	0.93
Cadereyta Jiménez	630	548	636	738	176	626	2,186	1.01
El Carmen	184	138	175	218	49	176	623	0.95
Hidalgo	92	70	97	131	26	98	357	1.05
Abasolo	3	1	6	12	1	5	23	2.00

```
orden = mun_tabla.index[::-1]
fig, ax = plt.subplots(figsize=(8.5, 6.4))
yy = np.arange(len(orden))
ax.hlines(yy, mun_tabla.loc[orden, "reg_q05"], mun_tabla.loc[orden, "reg_q95"], color=OXI
ax.hlines(yy, mun_tabla.loc[orden, "pers_q05"], mun_tabla.loc[orden, "pers_q95"], color=A
ax.scatter(mun_tabla.loc[orden, "2024"], yy, color=GRAFITO, s=22, zorder=3, label="2024 o
ax.set_yticks(yy); ax.set_yticklabels(orden)
ax.set_xscale("log")
ax.set_xlim(left=1)
ax.set_xticks([1, 10, 100, 1_000, 10_000, 100_000])
ax.set_xticklabels(["1", "10", "100", "1 mil", "10 mil", "100 mil"])
ax.xaxis.set_minor_formatter(NullFormatter())
ax.set_title("Por municipio: 2027 contra 2024")
ax.legend(loc="lower right")
pulir(ax); plt.tight_layout(); plt.show()
```

Por municipio: 2027 contra 2024



Pronóstico 2027 por municipio contra lo observado en 2024, en escala logarítmica. Segmento grueso azul: intervalo del 90 % por persistencia; segmento fino ocre: el mismo intervalo con cambio de régimen. Punto gris: 2024 observado.

Por persistencia, ningún municipio se mueve más de un 12 % respecto a 2024 (Abasolo aparte: con tres accidentes, cualquier cociente es ruido) y la mayoría queda entre 0.95 y 1.08: es el nivel de 2024 con suavizado. Los municipios chicos (Abasolo, Hidalgo, El Carmen) tienen los intervalos relativos más

anchos, porque son pocas celdas y el modelo lo sabe. El escenario de régimen es la misma imagen para todos: un intervalo de ~÷3 a ~×3.5 alrededor del nivel actual, porque σ_{rw} es común a los municipios. No hay un municipio "más propenso a saltar" que otro en el modelo; eso es un dato que no está en los seis años, o al menos no de forma que el modelo lo distinga de una racha.

---

5. Por celda

Aquí está lo que el modelo sí sabe hacer bien: **dónde**. El orden de las celdas no depende del escenario — el régimen municipal multiplica a todas las celdas de un municipio por igual — así que el mapa de 2027 es el mismo bajo persistencia y bajo régimen; lo que cambia es la escala.

```
celda["pred_q05"], celda["pred_q50"], celda["pred_q95"] = np.percentile(y_27, [5, 50, 95]
celda["lambda_med"] = np.median(lam_27, axis=1)
celda["lambda_media"] = lam_27.mean(axis=1)
celda["reg_q05"], celda["reg_q95"] = np.percentile(y_27r, [5, 95], axis=1)
celda["p_sube"] = (y_27 > celda.y_2024.values[:, None]).mean(axis=1)

# Probabilidad de que la celda esté en el 5 % superior: se ordena en cada draw y se cuent
k = int(N_HEX * 0.05)
rango = np.argsort(np.argsort(-lam_27, axis=0), axis=0)
celda["p_top5"] = (rango < k).mean(axis=1)

rho = stats.spearmanr(celda.lambda_med, celda.y_2024).correlation
top = celda.nlargest(k, "lambda_med")
print(f"Spearman entre el pronóstico 2027 y lo observado en 2024: {rho:.3f}")
print(f"Las {k} celdas del 5 % superior concentran el {top.lambda_med.sum() / celda.lambd
      f"(en 2024 tuvieron el {top.y_2024.sum() / celda.y_2024.sum():.1%} de lo observado)
print(f"Celdas con P(top 5 %) > 0.9: {(celda.p_top5 > 0.9).sum()}   |   entre 0.1 y 0.9:
```

```
Spearman entre el pronóstico 2027 y lo observado en 2024: 0.956
Las 113 celdas del 5 % superior concentran el 43.0% del pronóstico (en 2024 tuvieron el
42.8% de lo observado)
Celdas con P(top 5 %) > 0.9: 91   |   entre 0.1 y 0.9: 44
```

```
Hp = H.merge(celda, on="hex")
fig, ax = plt.subplots(figsize=(8.5, 8))
Hp.plot(column="lambda_med", cmap=SECUENCIAL, norm=LogNorm(vmin=0.3, vmax=Hp.lambda_med.m
        linewidth=0, legend=True, legend_kwds={"shrink": 0.5, "label": "accidentes espera
barra = fig.axes[-1]
barra.set_yticks([1, 10, 100]); barra.set_yticklabels(["1", "10", "100"]); barra.yaxis.se
mapa(ax, "Dónde: el pronóstico 2027 por celda")
plt.tight_layout(); plt.show()
```

Dónde: el pronóstico 2027 por celda



Pronóstico 2027 por celda (mediana de λ, escala logarítmica). Es el mapa de 2024 suavizado por el municipio y las vecinas; los corredores son los mismos que vio el análisis espacial.

```
fig, ax = plt.subplots(figsize=(8.5, 7))
H.plot(color=GRIS, ax=ax, linewidth=0)
cabeza = Hp[Hp.p_top5 >= 0.02]
cabeza.plot(column="p_top5", cmap=LinearSegmentedColormap.from_list("gs_top", [GRIS, OXID
            ax=ax, linewidth=0, legend=True, legend_kwds={"shrink": 0.5, "label": "P(celd
x0, y0, x1, y1 = cabeza.total_bounds
ax.set_xlim(x0 - 2_000, x1 + 2_000); ax.set_ylim(y0 - 2_000, y1 + 2_000)
mapa(ax, "Dónde está seguro el modelo", escala_km=5)
plt.tight_layout(); plt.show()
```

Dónde está seguro el modelo



Probabilidad a posteriori de que la celda esté en el 5 % superior en 2027, sobre la huella de la rejilla en gris (acercamiento al centro metropolitano). En rojo, las que están seguro; en el degradado, las que compiten por la frontera.

El primer mapa es el que se esperaba: Spearman de 0.96 con 2024, y el 5 % superior sigue concentrando el 43 % del total — el PAI que ningún modelo movió. El segundo es el que no daba el histórico: 89 celdas están en la cabeza con probabilidad mayor que 0.9, y otras 44 *compiten* por los 23 lugares que quedan. Si la decisión es "en qué 112 celdas intervenir", las seguras son las seguras; las de la frontera son donde el orden es una apuesta, y donde una covariable externa (exposición vial, uso de suelo) sí podría cambiar la lista.

```
top.assign(municipio=top.municipio.str.slice(0, 22))[
    ["hex", "municipio", "y_2024", "pred_q05", "pred_q50", "pred_q95", "p_top5"]
```

```
].head(15).style.format({"pred_q05": "{:.0f}", "pred_q50": "{:.0f}", "pred_q95": "{:.0f}"
```

hex	municipio	y_2024	pred_q05	pred_q50	pred_q95	p_top5
65_51	Monterrey	722	403	703	1147	1.00
60_55	Monterrey	686	357	624	1021	1.00
65_53	Monterrey	524	332	580	953	1.00
60_53	Monterrey	552	323	569	934	1.00
65_52	Monterrey	530	302	531	872	1.00
60_52	Monterrey	504	298	526	862	1.00
61_55	Monterrey	495	295	517	849	1.00
68_52	Monterrey	670	283	499	811	1.00
64_54	Monterrey	488	279	490	810	1.00
51_52	Santa Catarina	167	250	461	797	1.00
66_54	Monterrey	421	255	448	740	1.00
65_59	San Nicolás de los Gar	379	246	430	704	1.00
68_63	San Nicolás de los Gar	353	244	424	696	1.00
69_52	Monterrey	517	240	421	692	1.00
61_54	Monterrey	446	233	409	672	1.00

6. El producto

Una fila por hexágono, con la geometría de la celda y lo que un lector necesita para usar el pronóstico sin rehacer el modelo. Va a `processed/` como todo lo derivado; se regenera renderizando este notebook.

Columna	Qué es
`hex`, `municipio`	Identificador de la celda y municipio dominante
`y_2024`	Observado en 2024, para comparar
`lambda_med`, `lambda_media`	Mediana y media a posteriori de $\mu_{i,2027}$ por persistencia
`pred_q05`, `pred_q50`, `pred_q95`	Intervalo predictivo del conteo, persistencia
`reg_q05`, `reg_q95`	El mismo intervalo admitiendo cambio de régimen (solo cuantiles)
`p_top5`	Probabilidad de estar en el 5 % superior en 2027

Columna	Qué es
`p_sube`	Probabilidad de que el conteo de 2027 supere el de 2024

```
salida = Hp[["hex", "municipio", "y_2024", "lambda_med", "lambda_media",
             "pred_q05", "pred_q50", "pred_q95", "reg_q05", "reg_q95", "p_top5", "p_sube"
salida[["pred_q05", "pred_q50", "pred_q95", "reg_q05", "reg_q95"]] = salida[["pred_q05",
salida.to_parquet(rutas.PRONOSTICO_2027_GEO, compression="zstd", index=False)
print(f"{rutas.PRONOSTICO_2027_GEO.relative_to(rutas.RAIZ)}: {len(salida):,} celdas, {sal
salida.drop(columns="geometry").describe().loc[["mean", "50%", "max"]].round(2)
```

```
data/processed/pronostico_2027_geo.parquet: 2,270 celdas, 0.3 MB en memoria
```

	y_2024	lambda_med	lambda_media	pred_q05	pred_q50	pred_q95	reg_q05	reg_q95	p_top5	p_s
mean	31.41	32.05	32.44	16.23	30.99	53.64	7.1	121.16	0.05	0.4
50%	8.00	7.72	7.86	2.00	7.00	15.00	1.0	31.00	0.00	0.4
max	722.00	726.74	733.26	403.00	703.00	1147.00	172.0	2656.00	1.00	1.0

Conclusiones

El pronóstico central para 2027 es ~73.6 mil accidentes en la ZMM, con un intervalo del 90 % de ±3 % — condicional a que ningún municipio cambie de régimen de reporte. Por municipio, es el nivel de 2024 con suavizado; por celda, es el mapa de 2024 encogido hacia su municipio y sus vecinas, con Spearman 0.96 respecto a lo observado.

Lo que el modelo no controla domina la incertidumbre real. Si se admite que los municipios sigan moviéndose como en 2019-2024, el 90 % del total a tres años va de 53 mil a 230 mil, y la caminata aleatoria sin reversión inclina la suma hacia arriba. Ese escenario no es un pronóstico — es la medida de cuánto depende cualquier cifra de 2027 de decisiones de reporte que no están en los datos.

Lo robusto es el *dónde*. El orden de las celdas no depende del escenario. El 5 % superior concentra el 43 % del total, la mayoría de esas celdas está en la cabeza con probabilidad > 0.9, y `p_top5` separa las seguras de las que compiten por la frontera. Ahí — no en el total — es donde este pronóstico sirve para decidir.

Advertencia metodológica

Tres años es mucho para un modelo cuyo único mecanismo temporal es una caminata aleatoria estimada con cinco pasos. El escenario de persistencia subestima la incertidumbre por construcción; el de régimen la mide con una distribución que el modelo ajustó a un periodo dominado por ampliaciones de cobertura, y que por eso cree que un salto ×5 es tan verosímil hacia 2027 como lo fue en 2022. Ninguno de los dos distingue reporte de siniestralidad. Quien use `pronostico_2027_geo.parquet` debe

leer `lambda_med` como “el nivel de 2024 proyectado”, `p_top5` como el producto principal, y el par `reg_q05`/`reg_q95` como el recordatorio de lo que no se sabe.